Ψηφιακά Συστήματα VLSI

4η Εργαστηριακή Άσκηση

Ομάδα 32: Νικόλαος Λοβέρδος (03120442)

Δημήτριος Μιχαλακόπουλος (03120229)

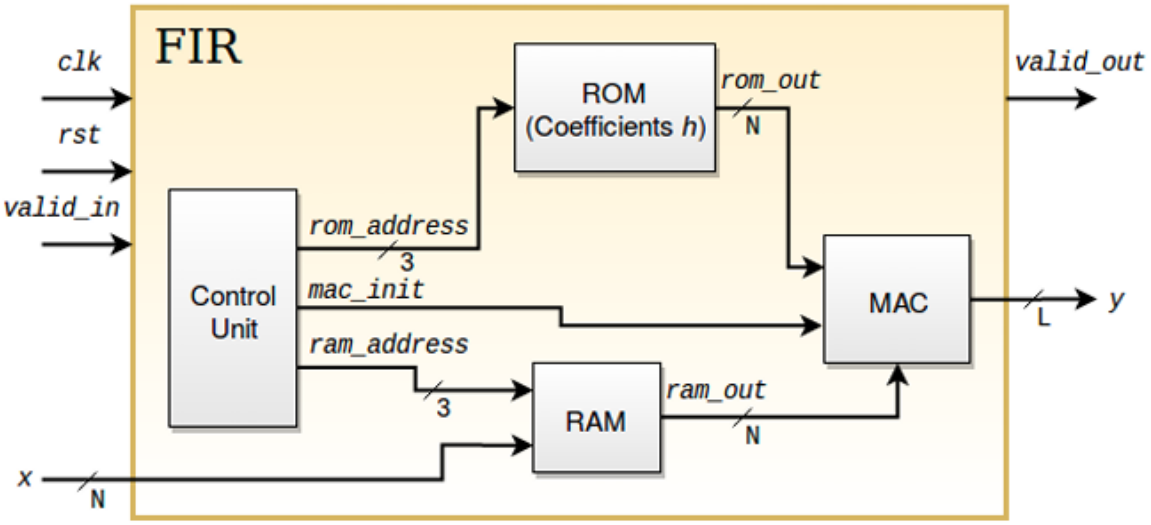
Βάρδια Γ: 11:30-12:30

Σχολή: Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Υλοποίηση FIR Φίλτρου

Στα πλαίσια αυτής της εργαστηριακής άσκησης μας ζητείται να σχεδιάσουμε ένα 8-tap FIR φίλτρο. Η υλοποίηση θέλουμε να γίνει για N=8 bits εύρος δεδομένων x και η αρχιτεκτονική που χρησιμοποιήσαμε φαίνεται στο παρακάτω σχηματικό:

$$y[n] = \sum_{k=0}^{M} h[k]\,x[n-k] = h[0]\,x[n] \;+\; h[1]\,x[n-1] \;+\; \ldots \;+\; h[M]\,x[n-M] \qquad (1)$$



Ακολουθεί ο κώδικας και ο σχολιασμός των βασικών modules του φίλτρου.

Control Unit

Ο κώδικας που υλοποιεί το Control Unit φαίνεται παρακάτω:

```vhdl
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity control_unit is
    port (clk, rst, valid_in : in std_logic;
          mac_init, we, valid_out : out std_logic;
          rom_addr, ram_addr : out std_logic_vector(2 downto 0)
    );
end control_unit;

architecture Behavioral of control_unit is

    signal count_reg : std_logic_vector (2 downto 0) := (others => '0');
    signal temp_valid_out : std_logic_vector (9 downto 0) := (others => '0');

begin

    process (clk)
    begin
        if rst='1' then
            count_reg <= (others => '0');
            mac_init <= '1';
            valid_out <= '0';
        elsif rising_edge(clk) then
            if count_reg = "000" then
                we <= valid_in;
                mac_init <= '1';
                temp_valid_out(0) <= valid_in;
            else
                we <= valid_in;
                mac_init <= '0';
                temp_valid_out(0) <= valid_in;
            end if;
        rom_addr <= count_reg;
        ram_addr <= count_reg;
        count_reg <= count_reg + 1;
        valid_out <= temp_valid_out(9);
        end if;
    end process;

    process (clk)
    begin
        if rising_edge(clk) then
            temp_valid_out(9 downto 1) <= temp_valid_out(8 downto 0);
        end if;
    end process;

end Behavioral;
```

Η υλοποίηση του Control Unit βασίζεται σε έναν μετρητή, ο οποίος μετράει 8 κύκλους ρολογιού και χρησιμοποιείται για την διευθυνσιοδότηση των μνημών ROM και RAM. Άμα βρισκόμαστε στον πρώτο κύκλο, τότε έχουμε νέο υπολογισμό και αρχικοποιούμε το MAC (mac_init=1), διαφορετικά περιμένουμε να τελειώσει ο προηγούμενος υπολογισμός. Στην συνέχεια, ανάλογα με την εγκυρότητα της εισόδου (valid_in), ενεργοποιείται η διαδικασία εγγραφής στην RAM (we) και ετοιμάζεται η τιμή της εγκυρότητας της εξόδου (valid_out). Για να πετύχουμε, όμως, τον κατάλληλο συγχρονισμό με την έξοδο, απαιτούνται 7 κύκλοι για να τελειώσει το μέτρημα του Control Unit, 1 κύκλος για να προσπελάσουμε τις μνήμες ROM και RAM και 1 κύκλος για τις πράξεις του MAC, άρα συνολικά το valid_out χρειάζεται καθυστέρηση 9 κύκλων ρολογιού. Τέλος, αυτή η μονάδα περιλαμβάνει ασύγχρονο reset (rst), το οποίο αρχικοποιεί τη μονάδα Control Unit, τη μνήμη RAM με '0' καθώς και το σήμα εγκυρότητας εξόδου valid_out.

ROM

Ο κώδικας που υλοποιεί την μνήμη ROM φαίνεται παρακάτω:

```
1   library IEEE;
2   use IEEE.std_logic_1164.all;
3   use IEEE.std_logic_unsigned.all;
4
5
6   entity mlab_rom is
7       generic (
8           coeff_width : integer :=8               --- width of coefficients (bits)
9       );
10      Port ( clk : in  std_logic;
11             en : in  std_logic;              --- operation enable
12             addr : in  std_logic_vector (2 downto 0);            -- memory address
13             rom_out : out  std_logic_vector (coeff_width-1 downto 0));   -- output data
14  end mlab_rom;
15
16  architecture Behavioral of mlab_rom is
17
18      type rom_type is array (7 downto 0) of std_logic_vector (coeff_width-1 downto 0);
19      signal rom : rom_type:= ("00001000", "00000111", "00000110", "00000101", "00000100", "00000011", "00000010",
20                               "00000001");                    -- initialization of rom with user data
21
22      signal rdata : std_logic_vector(coeff_width-1 downto 0) := (others => '0');
23  begin
24
25      rdata <= rom(conv_integer(addr));
26
27      process (clk)
28      begin
29          if (clk'event and clk = '1') then
30              if (en = '1') then
31                  rom_out <= rdata;
32              end if;
33          end if;
34      end process;
35
36  end Behavioral;
```

Σε αυτό το module είναι αποθηκευμένοι οι 8 συντελεστές του φίλτρου. Έτσι, ανάλογα με την τιμή rom_addr που έρχεται από το Control Unit, η ROM δίνει την αντίστοιχη έξοδο rom_out στην μονάδα MAC, η οποία κυμαίνεται από '1' εώς '8'.

RAM

Ο κώδικας που υλοποιεί την μνήμη RAM φαίνεται παρακάτω:

```
1   library IEEE;
2   use IEEE.std_logic_1164.all;
3   use IEEE.std_logic_unsigned.all;
4
5   entity mlab_ram is
6       generic (
7          data_width : integer :=8                --- width of data (bits)
8       );
9       port (clk  : in std_logic;
10            rst  : in std_logic;
11            we   : in std_logic;                                    --- memory write enable
12            en   : in std_logic;                                    --- operation enable
13            addr : in std_logic_vector(2 downto 0);                 -- memory address
14            di   : in std_logic_vector(data_width-1 downto 0);      -- input data
15            do   : out std_logic_vector(data_width-1 downto 0));    -- output data
16  end mlab_ram;
17
18  architecture Behavioral of mlab_ram is
19
20      type ram_type is array (7 downto 0) of std_logic_vector (data_width-1 downto 0);
21      signal RAM : ram_type := (others => (others => '0'));
22
23  begin
24
25      process (clk)
26      begin
27          if rst = '1' then
28              RAM <= (others => (others => '0'));
29          elsif clk'event and clk = '1' then
30              if en = '1' then
31                  if we = '1' then                -- write operation
32                      RAM(7 downto 1) <= RAM(6 downto 0);
33                      RAM(0) <= di;
34                      do <= di;
35                  else                        -- read operation
36                      do <= RAM(conv_integer(addr));
37                  end if;
38              end if;
39          end if;
40      end process;
41
42  end Behavioral;
```

Σε αυτό το module αποθηκεύεται η παρούσα τιμή της εισόδου x, καθώς και οι 7 προηγούμενες τιμές της. Αν είμαστε σε λειτουργία εγγραφής (we=1), τότε η νέα τιμή της εισόδου x αποθηκεύεται στην πρώτη θέση της μνήμης RAM και οι ήδη αποθηκευμένες 8 τιμές ολισθαίνουν κατά μία θέση με αποτέλεσμα η παλαιότερη χρονικά να διαγράφεται. Αλλιώς, αν είμαστε σε λειτουργία ανάγνωσης (we=0), ανάλογα με την τιμή ram_addr που έρχεται από το Control Unit, η RAM δίνει την αντίστοιχη έξοδο ram_out στην μονάδα MAC.

MAC

Ο κώδικας που υλοποιεί το MAC unit φαίνεται παρακάτω:

```
1   library IEEE;
2   use IEEE.std_logic_1164.all;
3   use IEEE.std_logic_unsigned.all;
4
5   entity mac is
6       port (clk, mac_init : in std_logic;
7             rom_out, ram_out : in std_logic_vector(7 downto 0);
8             y : out std_logic_vector(18 downto 0)
9       );
10  end mac;
11
12  architecture Behavioral of mac is
13
14      signal result : std_logic_vector(18 downto 0) := (others => '0');
15
16  begin
17
18      process (clk)
19      begin
20          if rising_edge(clk) then
21              if mac_init = '1' then
22                  result <= (others => '0');
23                  result(15 downto 0) <= rom_out * ram_out;
24              else
25                  result <= result + (rom_out * ram_out);
26              end if;
27          y <= result;
28          end if;
29      end process;
30
31  end Behavioral;
```

Σε αυτό το module γίνεται ο υπολογισμός της εξόδου y. Στην περίπτωση που έχουμε αρχικοποίηση του MAC (mac_init=1), δηλαδή το Control Unit είναι στον πρώτο κύκλο, το αποτέλεσμα ισούται με το γινόμενο της εξόδου της ROM επί το γινόμενο της εξόδου της RAM. Διαφορετικά, για τους επόμενους 7 κύκλους του ρολογιού, το γινόμενο rom_out επί ram_out προστίθεται στο ήδη υπάρχον αποτέλεσμα. Τέλος, για να αποφύγουμε την περίπτωση υπερχείλισης, το πλήθος των L bits της εξόδου y θα είναι:

$$L = length(rom_out) + length(ram_out) + ceil(log_2(M)) = 8 + 8 + 3 = 19$$

FIR

Ο κώδικας που υλοποιεί το FIR φίλτρο φαίνεται παρακάτω:

```
1   library IEEE;
2   use IEEE.std_logic_1164.all;
3   use IEEE.std_logic_unsigned.all;
4
5   entity fir is
6       port (clk, rst, valid_in : in std_logic;
7             x : in std_logic_vector(7 downto 0);
8             valid_out : out std_logic;
9             y : out std_logic_vector(18 downto 0)
10      );
11  end fir;
12
13  architecture Structural of fir is
14
15      component control_unit is
16          port (clk, rst, valid_in : in std_logic;
17                mac_init, we, valid_out : out std_logic;
18                rom_addr, ram_addr : out std_logic_vector(2 downto 0)
19          );
20      end component;
21
22      component mac is
23          port (clk, mac_init : in std_logic;
24                rom_out, ram_out : in std_logic_vector(7 downto 0);
25                y : out std_logic_vector(18 downto 0)
26          );
27      end component;
28
```

```
    component mlab_rom is
        generic (
           coeff_width : integer :=8
        );
        port ( clk : in  std_logic;
               en : in  std_logic;
               addr : in  std_logic_vector (2 downto 0);
               rom_out : out  std_logic_vector (coeff_width-1 downto 0));
    end component;

    component mlab_ram is
        generic (
           data_width : integer :=8
        );
        port (clk  : in std_logic;
              rst  : in std_logic;
              we   : in std_logic;
              en   : in std_logic;
              addr : in std_logic_vector(2 downto 0);
              di   : in std_logic_vector(data_width-1 downto 0);
              do   : out std_logic_vector(data_width-1 downto 0));
    end component;

    signal mac_init, mac_init_reg, we : std_logic;
    signal rom_addr, ram_addr : std_logic_vector(2 downto 0);
    signal x_reg, rom_out, ram_out : std_logic_vector(7 downto 0);

begin
    CU: control_unit port map (clk=>clk, rst=>rst, valid_in=>valid_in, mac_init=>mac_init,
    we=>we, valid_out=>valid_out, rom_addr=>rom_addr, ram_addr=>ram_addr);
    ROM: mlab_rom port map (clk=>clk, en=>'1', addr=>rom_addr, rom_out=>rom_out);
    RAM: mlab_ram port map (clk=>clk, rst=>rst, en=>'1', we=>we, addr=>ram_addr, di=>x_reg, do=>ram_out);
    MAC0: mac port map (clk=>clk, rom_out=>rom_out, ram_out=>ram_out, mac_init=>mac_init_reg, y=>y);

    process (clk)
    begin
        if rising_edge(clk) then
            x_reg <= x;
            mac_init_reg <= mac_init;
        end if;
    end process;

end Structural;
```

Το συνολικό κύκλωμα του FIR φίλτρου προκύπτει χρησιμοποιώντας τα modules που αναλύσαμε παραπάνω ως components. Επιπλέον, καθυστερούμε την είσοδο x για 1 κύκλο ρολογιού προκειμένου να συγχρονιστεί η έξοδος του Control Unit με την είσοδο της μνήμης RAM, και της αρχικοποίηση του MAC (mac_init) για 1 κύκλο ρολογιού προκειμένου να συγχρονιστεί η έξοδος της μνήμης ROM και της μνήμης RAM με την είσοδο του MAC unit. Έτσι, ολοκληρώνεται η υλοποίησή μας.

Το testbench που εξετάζει την ορθή λειτουργία του κυκλώματος είναι το εξής:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;
use IEEE.NUMERIC_STD.ALL;


entity FIR_tb is
end FIR_tb;

architecture Behavioral of FIR_tb is
    component fir is
    port (clk, rst, valid_in : in std_logic;
          x : in std_logic_vector(7 downto 0);
          valid_out : out std_logic;
          y : out std_logic_vector(18 downto 0)
    );
    end component;

    signal clk_tb : std_logic := '0';
    signal rst_tb : std_logic := '1';
    signal x_tb : std_logic_vector(7 downto 0) := (others => '0');
    signal y_tb : std_logic_vector(18 downto 0);
    signal valid_out_tb : std_logic;
    signal valid_in_tb: std_logic := '0';
    constant CLK_PERIOD : time := 10 ns;

begin
    DUT: fir port map (
        clk => clk_tb,
        rst => rst_tb,
        x => x_tb,
        y => y_tb,
        valid_in => valid_in_tb,
        valid_out => valid_out_tb
    );

    clk_process : process
    begin
        while now < 10000 ns loop
            clk_tb <= '0';
            wait for CLK_PERIOD/2;
            clk_tb <= '1';
            wait for CLK_PERIOD/2;
        end loop;
        wait;
    end process;

    validin_process : process
    begin
```

```
        while now < 10000 ns loop
            valid_in_tb <= '1';
            wait for 1*CLK_PERIOD;
            valid_in_tb <= '0';
            wait for 7*CLK_PERIOD;
        end loop;
    end process;

    stim_process : process
        variable x_data : std_logic_vector(7 downto 0);
    begin
        rst_tb <= '0';

        x_data := std_logic_vector(to_unsigned(208,8));
        x_tb <= x_data;
        wait for 8*CLK_PERIOD ;
        x_data := std_logic_vector(to_unsigned(231,8));
        x_tb <= x_data;
        wait for 8*CLK_PERIOD;
        x_data := std_logic_vector(to_unsigned(32,8));
        x_tb <= x_data;
        wait for 8*CLK_PERIOD;
        x_data := std_logic_vector(to_unsigned(233,8));
        x_tb <= x_data;
        wait for 8*CLK_PERIOD ;
        x_data := std_logic_vector(to_unsigned(161,8));
        x_tb <= x_data;
        wait for 8*CLK_PERIOD;
        x_data := std_logic_vector(to_unsigned(24,8));
        x_tb <= x_data;
        wait for 8*CLK_PERIOD;
        x_data := std_logic_vector(to_unsigned(71,8));
        x_tb <= x_data;
        wait for 8*CLK_PERIOD ;
        x_data := std_logic_vector(to_unsigned(140,8));
        x_tb <= x_data;
        wait for 8*CLK_PERIOD;
        x_data := std_logic_vector(to_unsigned(245,8));
        x_tb <= x_data;
        wait for 8*CLK_PERIOD;
        x_data := std_logic_vector(to_unsigned(247,8));
        x_tb <= x_data;
        wait for 8*CLK_PERIOD ;
        x_data := std_logic_vector(to_unsigned(40,8));
        x_tb <= x_data;
        wait for 8*CLK_PERIOD;
        x_data := std_logic_vector(to_unsigned(248,8));
        x_tb <= x_data;
```

```
98          wait for 8*CLK_PERIOD;
99          x_data := std_logic_vector(to_unsigned(245,8));
100         x_tb <= x_data;
101         wait for 8*CLK_PERIOD ;
102         x_data := std_logic_vector(to_unsigned(124,8));
103         x_tb <= x_data;
104         wait for 8*CLK_PERIOD;
105         x_data := std_logic_vector(to_unsigned(204,8));
106         x_tb <= x_data;
107         wait for 8*CLK_PERIOD;
108         x_data := std_logic_vector(to_unsigned(36,8));
109         x_tb <= x_data;
110         wait for 8*CLK_PERIOD ;
111         x_data := std_logic_vector(to_unsigned(107,8));
112         x_tb <= x_data;
113         wait for 8*CLK_PERIOD;
114         x_data := std_logic_vector(to_unsigned(234,8));
115         x_tb <= x_data;
116         wait for 8*CLK_PERIOD;
117         x_data := std_logic_vector(to_unsigned(202,8));
118         x_tb <= x_data;
119         wait for 8*CLK_PERIOD;
120         x_data := std_logic_vector(to_unsigned(245,8));
121         x_tb <= x_data;
122         wait for 8*CLK_PERIOD;
123         x_data := std_logic_vector(to_unsigned(0,8));
124         x_tb <= x_data;
125         wait for 8*CLK_PERIOD;
126
127         wait;
128     end process stim_process;
129
130 end Behavioral;
```

Το παραπάνω testbench εφαρμόζει στο FIR φίλτρο την παρακάτω ακολουθία εισόδων προκειμένου να εξετάσουμε τις αντίστοιχες εξόδους του:

208 231 32 233 161 24 71 140 245 247 40 248 245 124 204 36 107 234 202 245 0

Επιλέγουμε μια τυχαία τιμή εισόδου x = 140, άρα θεωρητικά ισχύουν τα εξής:

Μνήμη ROM: 1 2 3 4 5 6 7 8

Μνήμη RAM: 140 71 24 161 233 32 231 208

Έξοδος y: 140 282 354 998 2163 2355 3972 5636

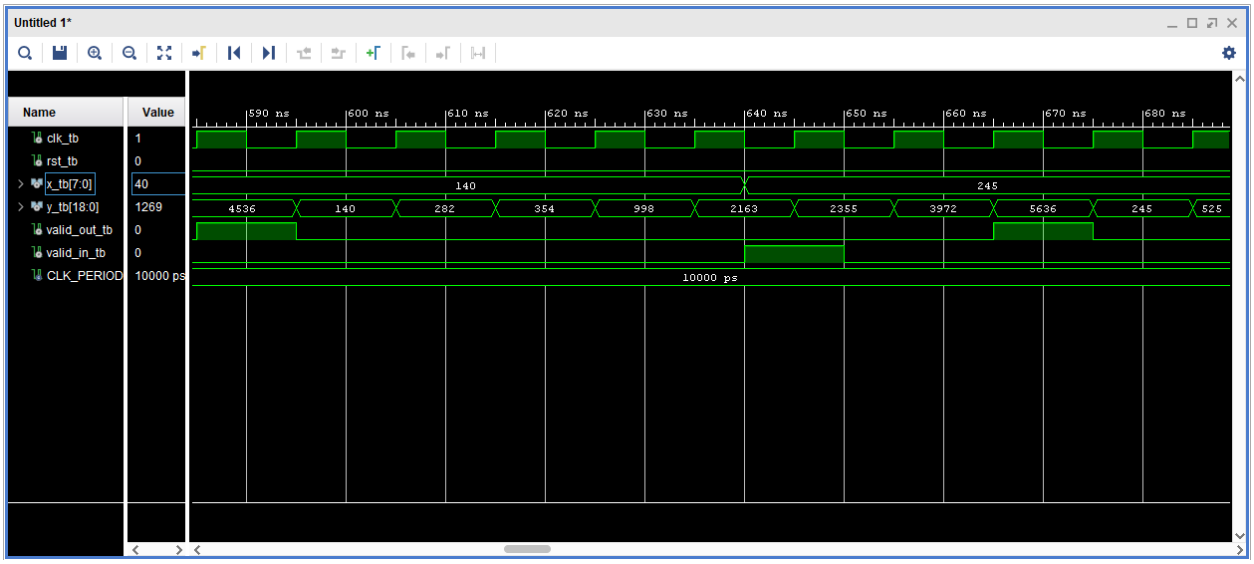
Επιλέγουμε άλλη μια τυχαία τιμή εισόδου x = 36, άρα πάλι θα ισχύουν τα εξής:

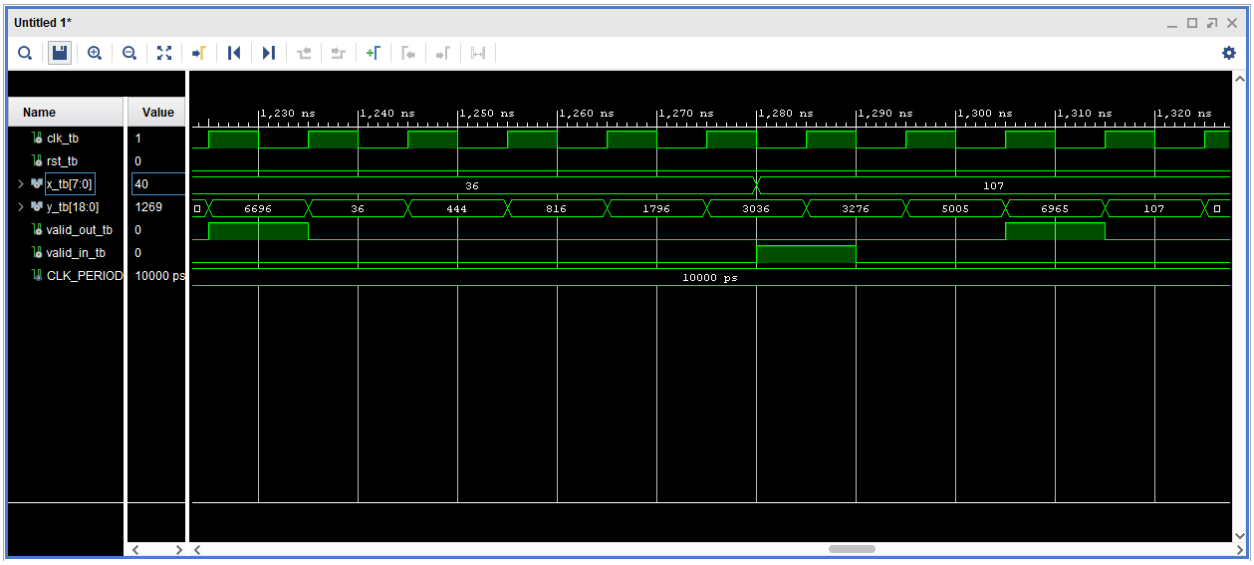
Μνήμη ROM: 1 2 3 4 5 6 7 8

Μνήμη RAM: 36 204 124 245 248 40 247 245

Έξοδος y: 36 444 816 1796 3036 3276 5005 6965

Η προσομοίωση του FIR φίλτρου φαίνεται παρακάτω:

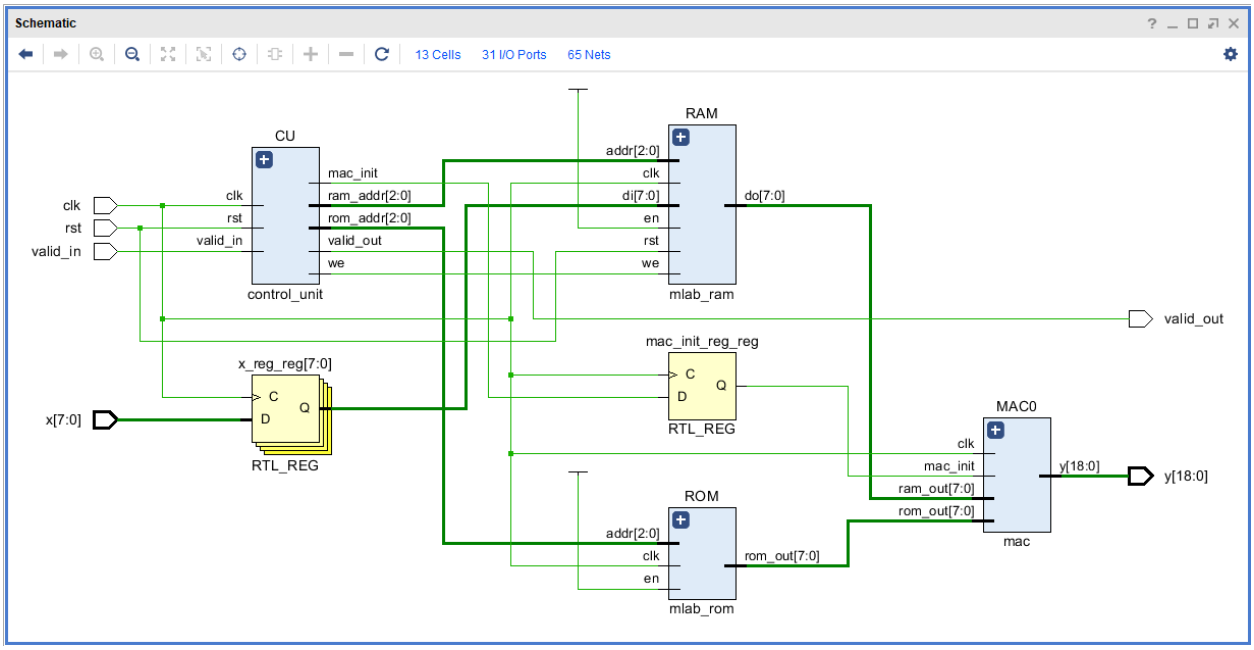




Παρατηρούμε ότι τα αποτελέσματα της προσομοίωσης για την έξοδο y του φίλτρου βρίσκονται σε πλήρη αντιστοιχία με τους θεωρητικούς υπολογισμούς

που πραγματοποιήσαμε παραπάνω. Επιπλέον, το valid_out γίνεται 1 μόνο όταν είμαστε στο τέλος του υπολογισμού, επομένως το FIR φίλτρο λειτουργεί ορθά.

Το RTL σχηματικό του FIR φίλτρου φαίνεται παρακάτω:



Το κρίσιμο μονοπάτι του FIR φίλτρου ξεκινάει από το ROM/rom_out_reg[1]/C και καταλήγει στην MAC0/result_reg[11]/D με συνολική καθυστέρηση 6.215 ns. Συνεπώς, η συχνότητα λειτουργίας του FPGA είναι τα 160.9 MHz.

Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
Path 1	∞	8	8	20	ROM/rom_out_reg[1]/C	MAC0/result_reg[11]/D	6.215	2.833	3.382
Path 2	∞	8	8	20	ROM/rom_out_reg[1]/C	MAC0/result_reg[10]/D	5.970	2.754	3.216
Path 3	∞	8	8	20	ROM/rom_out_reg[1]/C	MAC0/result_reg[9]/D	5.921	2.845	3.076
Path 4	∞	8	8	20	ROM/rom_out_reg[1]/C	MAC0/result_reg[8]/D	5.796	2.721	3.075
Path 5	∞	7	7	20	ROM/rom_out_reg[1]/C	MAC0/result_reg[7]/D	5.733	2.351	3.382
Path 6	∞	9	9	20	ROM/rom_out_reg[1]/C	MAC0/result_reg[17]/D	5.547	2.483	3.064
Path 7	∞	9	9	20	ROM/rom_out_reg[1]/C	MAC0/result_reg[18]/D	5.455	2.391	3.064
Path 8	∞	9	9	20	ROM/rom_out_reg[1]/C	MAC0/result_reg[16]/D	5.434	2.370	3.064
Path 9	∞	8	8	20	ROM/rom_out_reg[1]/C	MAC0/result_reg[13]/D	5.433	2.369	3.064
Path 10	∞	8	8	20	ROM/rom_out_reg[1]/C	MAC0/result_reg[15]/D	5.414	2.350	3.064

Τα resources που χρησιμοποιεί το FIR φίλτρο φαίνονται παρακάτω:

Resource	Estimation	Available	Utilization %
LUT	85	17600	0.48
LUTRAM	1	6000	0.02
FF	133	35200	0.38
IO	31	100	31.00
BUFG	1	32	3.13